

IE437 Data-Driven Decision Making and Control

2023 Spring Semester

1. Optimization Problem Modeling

Jinkyoo Park
Associate Professor
Industrial and Systems Engineering Department

Introduction

Motivation

- Mathematical optimization problems in standard form

$$\begin{aligned} \min_{x \in D} f(x) \\ \text{s. t. } g_i(x) \leq 0, \quad \forall i \in \{1, \dots, m\} \\ h_j(x) = 0, \quad \forall j \in \{1, \dots, p\} \end{aligned}$$

- $x \in \mathbb{R}^n$ is the optimization variable
- $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the objective of cost function
- $g_i : \mathbb{R}^n \rightarrow \mathbb{R}, \forall i \in \{1, \dots, m\}$, are the inequality constraint functions
- $h_j : \mathbb{R}^n \rightarrow \mathbb{R}, \forall j \in \{1, \dots, p\}$, are the equality constraint functions
- The **optimal value** p^* is defined as:

$$p^* := \inf \left\{ f(x) \left| \begin{array}{l} g_i(x) \leq 0, \quad \forall i \in \{1, \dots, m\} \\ h_j(x) = 0, \quad \forall j \in \{1, \dots, p\} \end{array} \right. \right\}$$

- $p^* = \infty$ if problem is infeasible (no $x \in D$ satisfies the constraints)
- $p^* = -\infty$ if problem is unbounded below

Motivation

- Mathematical optimization problems in standard form

$$\begin{aligned} \min_{x \in D} f(x) \\ \text{s. t. } g_i(x) \leq 0, \quad \forall i \in \{1, \dots, m\} \\ h_j(x) = 0, \quad \forall j \in \{1, \dots, p\} \end{aligned}$$

- x is **feasible** if $x \in D$ satisfies the constraints
- A feasible x is **optimal** if $f(x) = p^*$
- A feasible x is **locally optimal** if there is an $\epsilon > 0$ such that

$$f(x) \leq f(y), \quad \forall y \in S$$

$$\text{where } S := \left\{ y \in D \left| \begin{array}{l} \|x - y\|_2 \leq \epsilon \\ g_i(y) \leq 0, \quad \forall i \in \{1, \dots, m\} \\ h_j(y) = 0, \quad \forall j \in \{1, \dots, p\} \end{array} \right. \right\}$$

Motivation

- Mathematical optimization problems in standard form

$$\begin{aligned} \min_{x \in D} f(x) \\ \text{s. t. } g_i(x) \leq 0, \quad \forall i \in \{1, \dots, m\} \\ h_j(x) = 0, \quad \forall j \in \{1, \dots, p\} \end{aligned}$$

- D is the **implicit constraint (domain)** of the problem

$$D := \text{dom } f \cap \prod_{i=1}^m \text{dom } g_i \cap \prod_{j=1}^p \text{dom } h_j$$

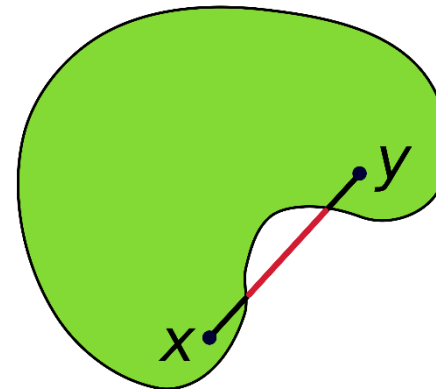
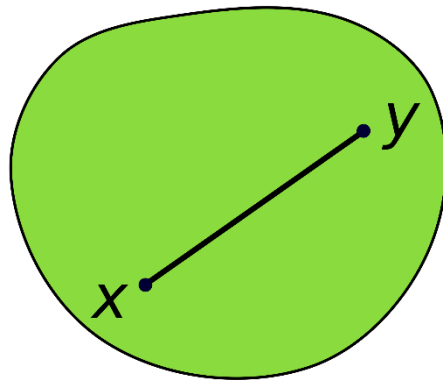
- The constraints $g_i(x) \leq 0$, and $h_j(x) = 0$ are the **explicit constraints**.
- A problem is **unconstrained** if it has no explicit constraints ($m = p = 0$)

Convex Optimization

Introduction

Convex set

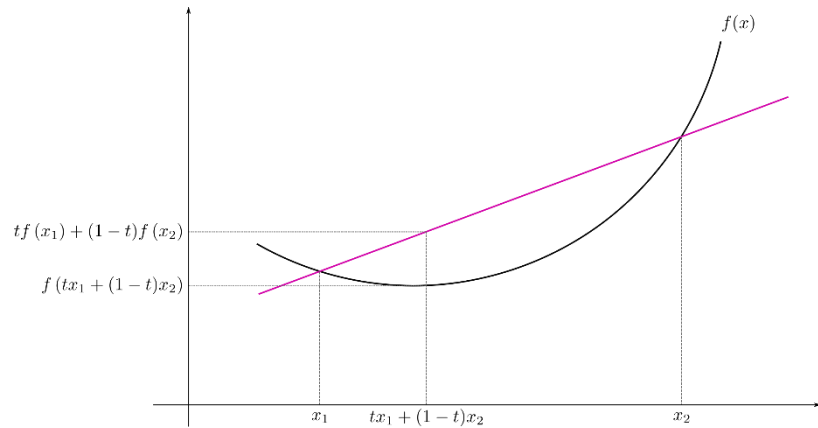
- A set of points in a Euclidean space is called a "convex set" if, for any two points in the set, the line segment connecting those two points lies entirely within the set. Put another way, a set is convex if it contains all the points on the straight line between any two points in the set.
- A subset C of S is **convex** if, for all x and y in C , the line segment connecting x and y is included in C . This means that the affine combination $(1 - t)x + ty$ belongs to C , for all x and y in C , and t in the interval $[0, 1]$.



Introduction

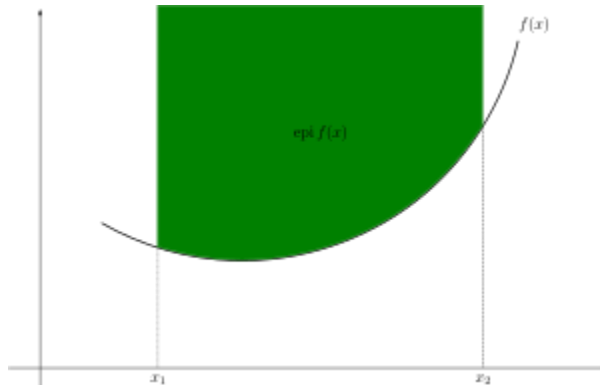
Convex function

- A real-valued function is called convex if the line segment between any two points on the graph of the function lies above the graph between the two points.



$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2)$$

- Equivalently, a function is convex if its epigraph (the set of points on or above the graph of the function) is a convex set.



Introduction

- Convex optimization problems in standard form

$$\begin{aligned} \min_{x \in D} & f(x) \\ \text{s. t. } & g_i(x) \leq 0, \quad \forall i \in \{1, \dots, m\} \\ & a_j^T x = b_j, \quad \forall j \in \{1, \dots, p\} \end{aligned}$$

- Convex optimization problems has **convex objective function** with **convex constraint set**.
- Convex objective function, that is, f is convex.
- Convex constraint set, that is,
 - ✓ Implicit constraint D is convex set
 - ✓ The inequality constraints are convex, that is, g_i 's are convex
 - ✓ The equality constraints $a_j^T x = b_j$'s are affine (often written as $\mathbf{A}x = \mathbf{b}$)

Local and global optima

Theorem 1.

Let the convex problem is $\min_{x \in D} f(x)$ where $f(x)$ is the objective function, and D is the constraint set. Then, *any local optimal point of a convex problem is (globally) optimal.*

Proof. Suppose x^* is local optimal, but not global optimal. Then, there exists $\hat{x}$ with $f(x^*) > f(\hat{x})$.

Since x^* is local optimal, there is an $\epsilon > 0$ such that

$$f(y) \geq f(x^*), \quad \forall y \in S = \{y \in D \mid \|y - x^*\|_2 \leq \epsilon\} \quad \dots\dots\dots (1)$$

Since D is convex set, there is $y = \theta \hat{x} + (1 - \theta)x^* \in S$ with $\theta = \frac{\epsilon}{2\|\hat{x} - x^*\|_2}$.

Since f is convex,

$$f(y) = f(\theta \hat{x} + (1 - \theta)x^*) \leq \theta f(\hat{x}) + (1 - \theta)f(x^*) < f(x^*) \quad \dots\dots\dots (2)$$

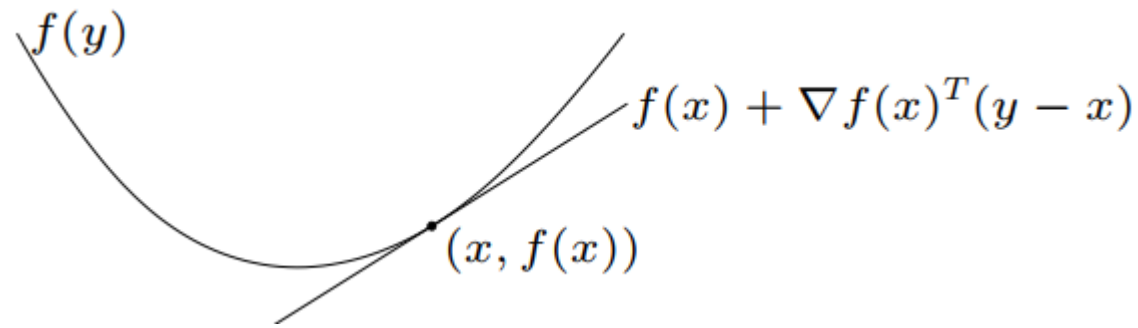
The equation (1) and (2) contradict our assumption that x^* is local optimal.

Optimality criterion

- Suppose f is differentiable (i.e., its gradient ∇f exists at each point in $\mathbf{dom} f$, which is open). Then f is convex if and only if $\mathbf{dom} f$ is convex and

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

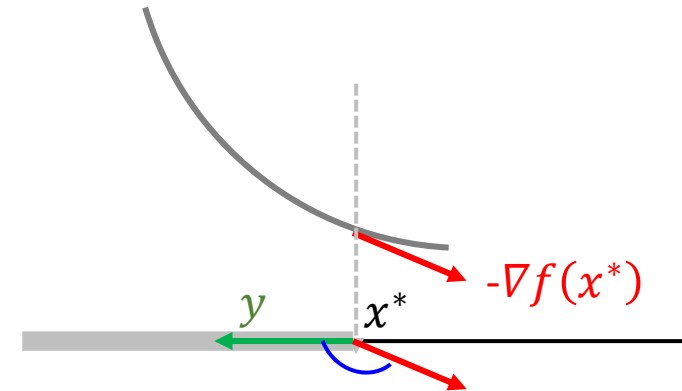
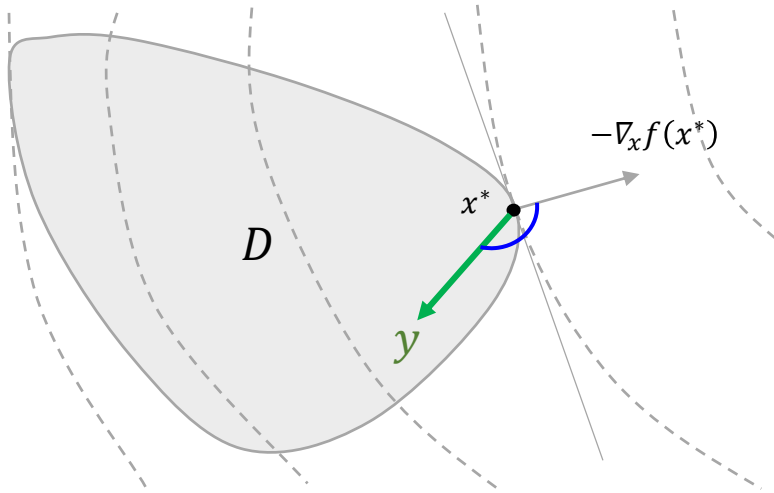
holds for all $x, y \in \mathbf{dom} f$



- For a convex function, the first-order Taylor approximation is a global local estimator of the function. Conversely, the first-order Taylor approximation is a always global underestimator of the function, then the function is convex
- Global information about a convex function can be derived from local information about the function
- e.g., if $\nabla f(x) = 0$, then $f(y) \geq f(x)$, i. e., x is a global minimizer.

Optimality criterion

- Let the convex problem is $\min_{x \in D} f(x)$ where $f(x)$ is the differentiable objective function, and D is the constraint set.
- x^* is optimal if and only if it is feasible ($x \in D$) and $\nabla f(x^*)^T (y - x^*) \geq 0, \forall y \in D$
(If nonzero, $\nabla f(x^*)$ define a supporting hyperplane to feasible set D at x^*)



Optimality criterion

- x^* is optimal if and only if
- **Unconstrained problem** $\min_{x \in D} f(x)$ where D is implicit constraint set:

$$x^* \in D, \quad \nabla_x f(x^*) = 0$$

- **Equality constrained problem** $\min_{x \in D \cap \{x | \mathbf{A}x = \mathbf{b}\}} f(x)$: There exists a λ such that

$$x^* \in D, \quad \mathbf{A}x^* = \mathbf{b}, \quad \nabla_x f(x^*) + \mathbf{A}^T \lambda = 0$$

✓ This is the classical Lagrange multiplier optimality condition

Equivalent convex problems

- Two problems are (informally) equivalent if the solution of one is readily obtained from the solution of the other, and vice-versa.
- **Eliminating equality constraints**

$$\begin{aligned} & \min_{x \in D} f(x) \\ & \text{s. t. } g_i(x) \leq 0, \quad \forall i \in \{1, \dots, m\} \\ & \quad \mathbf{Ax} = \mathbf{b} \end{aligned}$$

is equivalent to

$$\begin{aligned} & \min_{z \in \{z \mid \mathbf{F}z + x_0 \in D\}} f(\mathbf{F}z + x_0) \\ & \text{s. t. } g_i(\mathbf{F}z + x_0) \leq 0, \quad \forall i \in \{1, \dots, m\} \end{aligned}$$

where $\mathbf{F}$ and x_0 are such that

$$\mathbf{Ax} = \mathbf{b} \leftrightarrow x = \mathbf{F}z + x_0$$

Equivalent convex problems

- Introducing equality constraints

$$\begin{aligned} \min_{x \in D} & f(\mathbf{A}_0 x + \mathbf{b}_0) \\ \text{s. t. } & g_i(\mathbf{A}_i x + \mathbf{b}_i) \leq 0, \quad \forall i \in \{1, \dots, m\} \end{aligned}$$

is equivalent to

$$\begin{aligned} \min_{z_0 \in \{\mathbf{A}_0 x + \mathbf{b}_0 \mid x \in D\}} & f(z_0) \\ \text{s. t. } & g_i(z_i) \leq 0, \quad \forall i \in \{1, \dots, m\} \\ & z_i = \mathbf{A}_i x + \mathbf{b}_i, \quad \forall i \in \{0, \dots, m\} \end{aligned}$$

Equivalent convex problems

- Introducing slack variables for linear inequalities

$$\begin{array}{ll} \min_{x \in D} & f(x) \\ \text{s. t.} & a_i^T x \leq b_i, \quad \forall i \in \{1, \dots, m\} \end{array}$$

is equivalent to

$$\begin{array}{ll} \min_{(x,s) \in D \times \mathbb{R}_+^m} & f(x) \\ \text{s. t.} & a_i^T x + s_i = b_i, \quad \forall i \in \{1, \dots, m\} \end{array}$$

Equivalent convex problems

- **Epigraph form:** standard form of convex problem

$$\begin{aligned} \min_{x \in D} \quad & f(x) \\ \text{s. t.} \quad & g_i(x) \leq 0, \quad \forall i \in \{1, \dots, m\} \\ & h_j(x) = 0, \quad \forall j \in \{1, \dots, p\} \end{aligned}$$

is equivalent to

$$\begin{aligned} \min_{(x,t) \in D \times \mathbb{R}} \quad & t \\ \text{s. t.} \quad & f(x) - t \leq 0 \\ & g_i(x) \leq 0, \quad \forall i \in \{1, \dots, m\} \\ & h_j(x) = 0, \quad \forall j \in \{1, \dots, p\} \end{aligned}$$

Equivalent convex problems

- Minimizing over some variables

$$\begin{aligned} \min_{(x_1, x_2) \in D_1 \times D_2} & f(x_1, x_2) \\ \text{s. t. } & g_i(x_1) \leq 0, & \forall i \in \{1, \dots, m\} \\ & h_j(x_1) = 0, & \forall j \in \{1, \dots, p\} \end{aligned}$$

is equivalent to

$$\begin{aligned} \min_{x_1 \in D_1} & \hat{f}(x_1) \\ \text{s. t. } & g_i(x_1) \leq 0, & \forall i \in \{1, \dots, m\} \\ & h_j(x_1) = 0, & \forall j \in \{1, \dots, p\} \end{aligned}$$

where $\hat{f}(x_1) = \inf_{x_2 \in D_2} f(x_1, x_2)$

Advantages of Convex Optimization

- Convex optimization provides a globally optimal solution
- Reliable and efficient solvers
- Specific solvers and internal parameters, e.g., initialization, step-size, batch-size does not matter
- We can check global optimality via KKT conditions
- Dual problem provides a lower-bound and an optimality gap
- Distributed and decentralized methods are well-studied

Input Convex Neural Networks

Brandon Amos¹ Lei Xu^{2*} J. Zico Kolter¹

Abstract

This paper presents the input convex neural network architecture. These are scalar-valued (potentially deep) neural networks with constraints on the network parameters such that the output of the network is a convex function of (some of) the inputs. The networks allow for efficient inference via optimization over some inputs to the network given others, and can be applied to settings including structured prediction, data imputation, reinforcement learning, and others. In this paper we lay the basic groundwork for these models, proposing methods for inference, optimization and learning, and analyze their representational power. We show that many existing neural network architectures can be made input-convex with a minor modification, and develop specialized optimization algorithms tailored to this setting. Finally, we highlight the performance of the methods on multi-label prediction, image completion, and reinforcement learning problems, where we show improvement over the existing state of the art in many cases.

1. Introduction

In this paper, we propose a new neural network architecture that we call the *input convex neural network* (ICNN). These are *scalar-valued* neural networks $f(x, y; \theta)$ where x and y denotes inputs to the function and θ denotes the parameters, built in such a way that the network is convex in (a subset of) inputs y .³ The fundamental benefit to these ICNNs is that we can *optimize* over the convex inputs to the network given some fixed value for other inputs. That is, given some fixed x (and possibly some fixed elements of

y) we can globally and efficiently (because the problem is convex) solve the optimization problem

$$\operatorname{argmin}_y f(x, y; \theta). \quad (1)$$

Fundamentally, this formalism lets us perform inference in the network via *optimization*. That is, instead of making predictions in a neural network via a purely feedforward process, we can make predictions by optimizing a scalar function (which effectively plays the role of an energy function) over some inputs to the function given others. There are a number of potential use cases for these networks.

Structured prediction As is perhaps apparent from our notation above, a key application of this work is in structured prediction. Given (typically high-dimensional) structured input and output spaces $\mathcal{X} \times \mathcal{Y}$, we can build a network over (x, y) pairs that encodes the energy function for this pair, following typical energy-based learning formalisms (LeCun et al., 2006). Prediction involves finding the $y \in \mathcal{Y}$ that minimizes the energy for a given x , which is exactly the argmin problem in (1). In our setting, assuming that $\mathcal{Y}$ is a convex space (a common assumption in structured prediction), this optimization problem is convex. This is similar in nature to the structured prediction energy networks (SPENs) (Belanger & McCallum, 2016), which also use deep networks over the input and output spaces, with the difference being that in our setting f is convex in y , so the optimization can be performed globally.

Data imputation Similar to structured prediction but slightly more generic, if we are given some space $\mathcal{Y}$ we can learn a network $f(y; \theta)$ (removing the additional x inputs, though these can be added as well) that, given an example with some subset $\mathcal{I}$ missing, imputes the likely values of these variables by solving the optimization problem as above $\hat{y}_{\mathcal{I}} = \operatorname{argmin}_{y_{\mathcal{I}}} f(y_{\mathcal{I}}, y_{\mathcal{I}^c}; \theta)$. This could be used

³We emphasize the term “input convex” since convexity in machine learning typically refers to convexity (of the loss minimization learning problem) in the *parameters*, which is not the case here. Note that in our notation, f needs only be a convex function in y , and may still be non-convex in the remaining inputs x . Training these neural networks remains a nonconvex problem, and the convexity is only being exploited at inference time.

OPTIMAL CONTROL VIA NEURAL NETWORKS: A CONVEX APPROACH

Yize Chen,^{*} Yuanyuan Shi,^{*} Baosen Zhang

Department of Electrical and Computer Engineering,
University of Washington,
Seattle, WA 98195, USA
{yizechen, yyshi, zhangbao}@uw.edu

ABSTRACT

Control of complex systems involves both system identification and controller design. Deep neural networks have proven to be successful in many identification tasks, however, from model-based control perspective, these networks are difficult to work with because they are typically nonlinear and nonconvex. Therefore many systems are still identified and controlled based on simple linear models despite their poor representation capability. In this paper we bridge the gap between model accuracy and control tractability faced by neural networks, by explicitly constructing networks that are convex with respect to their inputs. We show that these input convex networks can be trained to obtain accurate models of complex physical systems. In particular, we design input convex recurrent neural networks to capture temporal behavior of dynamical systems. Then optimal controllers can be achieved via solving a convex model predictive control problem. Experiment results demonstrate the good potential of the proposed input convex neural network based approach in a variety of control applications. In particular we show that in the MuJoCo locomotion tasks, we could achieve over 10% higher performance using $5 \times$ less time compared with state-of-the-art model-based reinforcement learning method; and in the building HVAC control example, our method achieved up to 20% energy reduction compared with classic linear models.

1 INTRODUCTION

Decisions on how to best operate and control complex physical systems such as the power grid, commercial and industrial buildings, transportation networks and robotic systems are of critical societal importance. These systems are often challenging to control because they tend to have complicated and poorly understood dynamics, sometimes with legacy components are built over a long period of time (Wolf, 2009). Therefore detailed models for these systems may not be available or may be intractable to construct. For instance, since buildings account for 40% of the global energy consumption (Cheng et al., 2008), many approaches have been proposed to operate buildings more efficiently by controlling their heating, ventilation, and air conditioning (HVAC) systems (Zhang et al., 2017). Most of these methods, however, suffer from two drawbacks. On one hand, a detailed physics model of a building can be used to accurately describe its behavior, but this model can take years to develop. On the other hand, simple control algorithms have been developed by using linear (RC circuit) models (Ma et al., 2012) to represent buildings, but the performance of these models may be poor since the building dynamics can be far from linear (Shaikh et al., 2014).

In this paper, we leverage the availability of data to strike a balance between requiring painstaking manual construction of physics based models and the risk of not capturing rich and complex system dynamics through models that are too simplistic. In recent years—with the growing deployment of sensors in physical and robotics systems—large amount of operational data have been collected, such as in smart buildings (Suryadevara et al., 2015), legged robotics (Meger et al., 2015) and manipulators (Deisenroth et al., 2011). Using these data, the system dynamics can be learned directly and then automatically updated at periodic intervals. One popular method is to parameterize these

^{*}Authors contribute equally.

¹Work done while author was at Carnegie Mellon University. ²School of Computer Science, Carnegie Mellon University. Pittsburgh, PA, USA ³Department of Computer Science and Technology, Tsinghua University. Beijing, China. Correspondence to: Brandon Amos <bamos@cs.cmu.edu>, J. Zico Kolter <zkolter@cs.cmu.edu>.

Convex Optimization in ML

Optimal Control Via Neural Networks: A Convex Approach

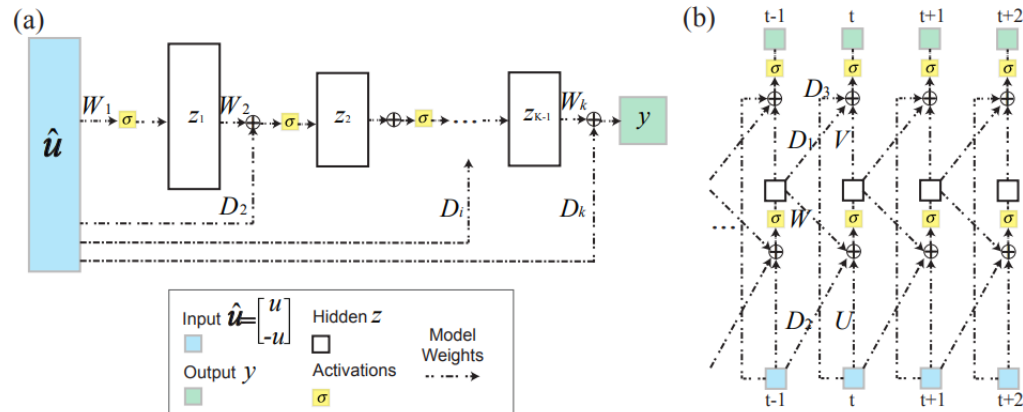


Figure 2: Input convex neural network. Input convex neural network. (a) Input convex feed-forward neural networks (ICNN). One notable addition is the direct “passthrough” layers $D_{2:k}$ that connect the inputs to hidden units for better model representation ability. (b) The proposed input convex recurrent neural networks (ICRNN) architectures. In our control settings, we keep all weights in both networks nonnegative, while expanding the inputs with $-u$.

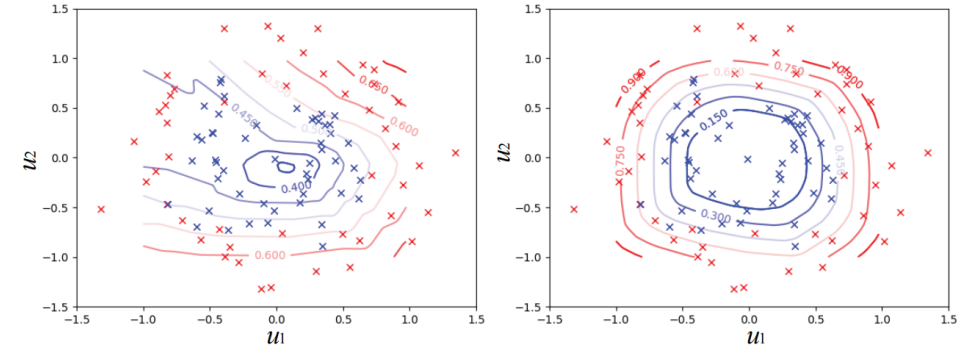


Figure 5: Toy example on classifying circle data with label 0 (blue cross) and label 1 (red cross) along with conventional neural networks (left) and ICNN (right) decision contour lines. A decision maker is interested in finding a u that has the highest probability of being labeled 0.

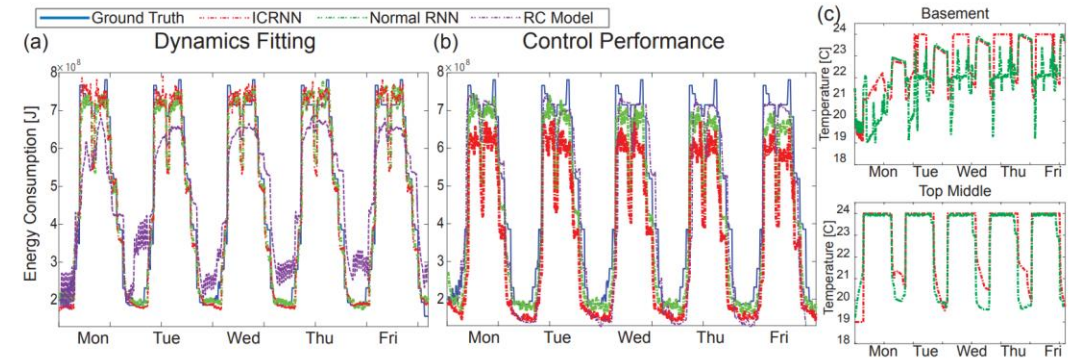


Figure 4: Results for constrained optimization of building energy management. (a) ICRNN is able to model the building dynamics as accurately as conventional RNN; (b) Compared to conventional RNN model, ICRNN finds control actions which lead to 11.52% more of energy savings, and (c) ICRNN provides stable control actions while decisions generated by conventional RNN vary dramatically.

Convex Optimization in ML

Differentiable Convex Optimization Layers

Akshay Agrawal
Stanford University
akshayka@cs.stanford.edu

Brandon Amos
Facebook AI
bda@fb.com

Shane Barratt
Stanford University
sbarratt@stanford.edu

Stephen Boyd
Stanford University
boyd@stanford.edu

Steven Diamond
Stanford University
diamond@cs.stanford.edu

J. Zico Kolter*
Carnegie Mellon University
Bosch Center for AI
zkolter@cs.cmu.edu

Abstract

Recent work has shown how to embed *differentiable optimization problems* (that is, problems whose solutions can be backpropagated through) as layers within deep learning architectures. This method provides a useful inductive bias for certain problems, but existing software for differentiable optimization layers is rigid and difficult to apply to new settings. In this paper, we propose an approach to differentiating through disciplined convex programs, a subclass of convex optimization problems used by domain-specific languages (DSLs) for convex optimization. We introduce *disciplined parametrized programming*, a subset of disciplined convex programming, and we show that every disciplined parametrized program can be represented as the composition of an affine map from parameters to problem data, a solver, and an affine map from the solver’s solution to a solution of the original problem (a new form we refer to as *affine-solver-affine* form). We then demonstrate how to efficiently differentiate through each of these components, allowing for end-to-end analytical differentiation through the entire convex program. We implement our methodology in version 1.1 of CVXPY, a popular Python-embedded DSL for convex optimization, and additionally implement differentiable layers for disciplined convex programs in PyTorch and TensorFlow 2.0. Our implementation significantly lowers the barrier to using convex optimization problems in differentiable programs. We present applications in linear machine learning models and in stochastic control, and we show that our layer is competitive (in execution time) compared to specialized differentiable solvers from past work.

1 Introduction

Recent work has shown how to differentiate through specific subclasses of convex optimization problems, which can be viewed as functions mapping problem data to solutions [6, 31, 10, 1, 4]. These layers have found several applications [40, 6, 35, 27, 5, 53, 75, 52, 12, 11], but many applications remain relatively unexplored (see, e.g., [4, §8]).

While convex optimization layers can provide useful inductive bias in end-to-end models, their adoption has been slowed by how difficult they are to use. Existing layers (e.g., [6, 1]) require users to transform their problems into rigid canonical forms by hand. This process is tedious, error-prone, and time-consuming, and often requires familiarity with convex analysis. Domain-specific languages (DSLs) for convex optimization abstract away the process of converting problems to canonical forms, letting users specify problems in a natural syntax; programs are then lowered to canonical forms and

* Authors listed in alphabetical order.

OptNet: Differentiable Optimization as a Layer in Neural Networks

Brandon Amos¹ J. Zico Kolter¹

Abstract

This paper presents OptNet, a network architecture that integrates optimization problems (here, specifically in the form of quadratic programs) as individual layers in larger end-to-end trainable deep networks. These layers encode constraints and complex dependencies between the hidden states that traditional convolutional and fully-connected layers often cannot capture. We explore the foundations for such an architecture: we show how techniques from sensitivity analysis, bilevel optimization, and implicit differentiation can be used to exactly differentiate through these layers and with respect to layer parameters; we develop a highly efficient solver for these layers that exploits fast GPU-based batch solves within a primal-dual interior point method, and which provides backpropagation gradients with virtually no additional cost on top of the solve; and we highlight the application of these approaches in several problems. In one notable example, the method is learned to play mini-Sudoku (4x4) given just input and output games, with no a-priori information about the rules of the game; this highlights the ability of OptNet to learn hard constraints better than other neural architectures.

1. Introduction

In this paper, we consider how to treat exact, constrained optimization as an individual layer within a deep learning architecture. Unlike traditional feedforward networks, where the output of each layer is a relatively simple (though non-linear) function of the previous layer, our optimization framework allows for individual layers to capture much richer behavior, expressing complex operations that in total can reduce the overall depth of the network while preserving richness of representation. Specifically, we build a

framework where the output of the $i + 1$ th layer in a network is the *solution* to a constrained optimization problem based upon previous layers. This framework naturally encompasses a wide variety of inference problems expressed within a neural network, allowing for the potential of much richer end-to-end training for complex tasks that require such inference procedures.

Concretely, in this paper we specifically consider the task of solving small quadratic programs as individual layers. These optimization problems are well-suited to capturing interesting behavior and can be efficiently solved with GPUs. Specifically, we consider layers of the form

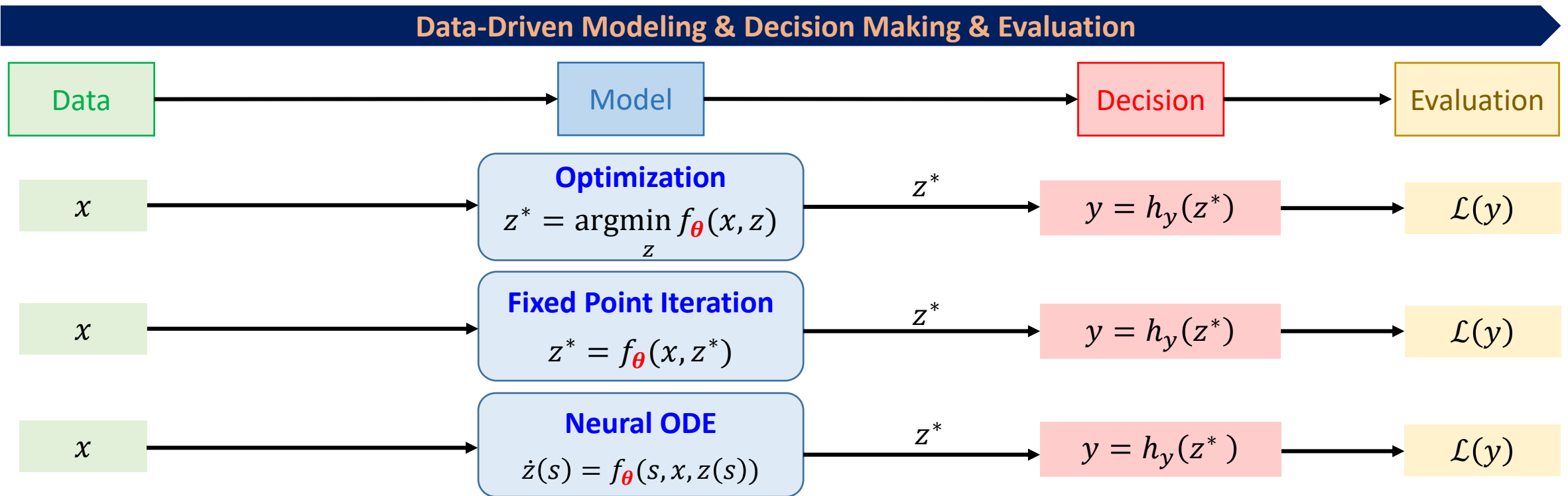
$$\begin{aligned} z_{i+1} = \underset{z}{\operatorname{argmin}} \quad & \frac{1}{2} z^T Q(z_i) z + q(z_i)^T z \\ \text{subject to} \quad & A(z_i) z = b(z_i) \\ & G(z_i) z \leq h(z_i) \end{aligned} \quad (1)$$

where z is the optimization variable, $Q(z_i)$, $q(z_i)$, $A(z_i)$, $b(z_i)$, $G(z_i)$, and $h(z_i)$ are parameters of the optimization problem. As the notation suggests, these parameters can depend in any differentiable way on the previous layer z_i , and which can eventually be optimized just like any other weights in a neural network. These layers can be learned by taking the gradients of some loss function with respect to the parameters. In this paper, we derive the gradients of (1) by taking matrix differentials of the KKT conditions of the optimization problem at its solution.

In order to make the approach practical for larger networks, we develop a custom solver which can simultaneously solve multiple small QPs in batch form. We do so by developing a custom primal-dual interior point method tailored specifically to dense batch operations on a GPU. In total, the solver can solve batches of quadratic programs over 100 times faster than existing highly tuned quadratic programming solvers such as Gurobi and CPLEX. One crucial algorithmic insight in the solver is that by using a specific factorization of the primal-dual interior point update, we can obtain a backward pass over the optimization layer virtually “for free” (i.e., requiring no additional factorization once the optimization problem itself has been solved). Together, these innovations enable parameterized optimization problems to be inserted within the architecture of existing deep networks.

¹School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, USA. Correspondence to: Brandon Amos <bamos@cs.cmu.edu>, J. Zico Kolter <zkolter@cs.cmu.edu>.

Convex Optimization in ML



- **Implicit Deep Learning or Declarative Neural Network** Parametrize mappings $\mathcal{X} \rightarrow \mathcal{Y}$ by specifying the *conditions* that we want the model's output to satisfy (i.e., **behavioral inductive biases**)
 - Training of implicit deep learning is conducted by computing the following gradient using **implicit function theorem**:
$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial z^*}{\partial \theta} \frac{\partial \mathcal{L}}{\partial z^*}$$

$\frac{\partial z^*}{\partial \theta}$: How model parameters θ affects the equilibrium point z^*
 - Gradient is computed pivoting upon the equilibrium point (i.e., solution). We have to find it using **numerical method**!

Data-Driven Bilevel Design Optimization with Input Convex Graph Neural Networks (On Going)

Target Problem

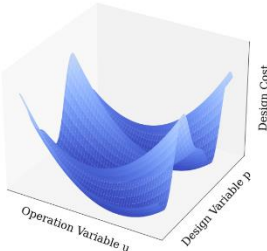
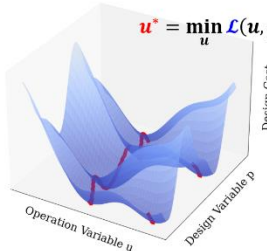
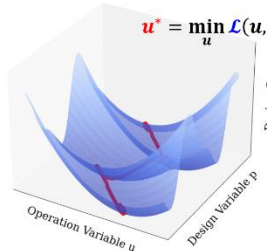
$$\begin{aligned} \min_p \quad & \sum_{t=0}^{T-1} \mathcal{L}(x_{t+1}, u_t^*; p) \\ \text{s.t. } \quad & u_{0:T-1}^* = \arg \min_{u_{0:T-1}} \sum_{t=0}^{T-1} \mathcal{L}(x_{t+1}, u_t; p) \\ & \text{s.t. } \boxed{x_{t+1} = f(x_t, u_t; p)} \quad \forall t \\ & \quad u_t \in \mathcal{U}_p, \quad \forall t \\ & p \in \mathcal{P} \end{aligned}$$

When the dynamic model is unknown

- Bilevel Design Optimization:
 - 1) upper-level design optimization
 - 2) several lower-level dynamic control problems
- Applied on many dynamical systems with design variables, such as wind farm, robotics, furnace, etc.

Data-Driven Bilevel Design Optimization with Input Convex Graph Neural Networks (On Going)

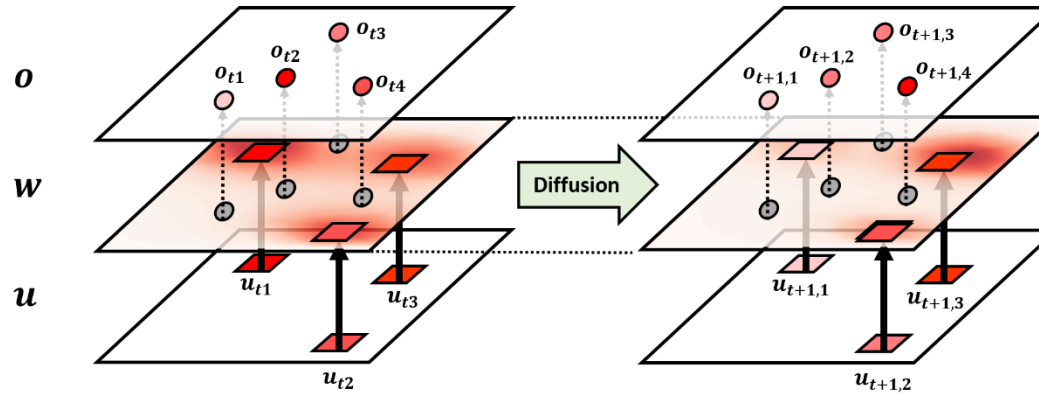
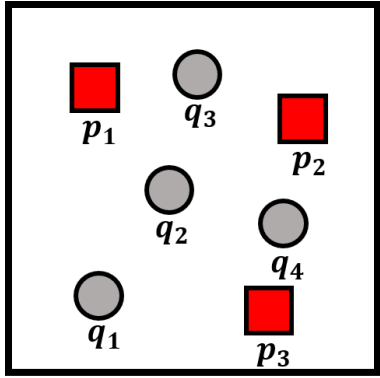
Motivation

	ICGNN + Single-Layer	GNN + Implicit	ICGNN + Implicit (Our Method)
Design Cost Surface			
Optimization Search Space	Operation & Design Variable Space $\mathcal{U} \times \mathcal{P}$	Design Variable Space $\mathcal{P}$	Design Variable Space $\mathcal{P}$
Solve Lower-Level Optimization (Find Optimal u^* with fixed p)	-	 Non-Convex Optimization	 Convex Optimization
Satisfy the Sufficiency of KKT	-	No	Yes

- Proposed Method: ICGNN Surrogate model + Implicit differentiation
 - Input Convex Graph Neural Networks (ICGNN): GNN which the output is a convex function of input
 - Balanced between model expressivity and optimization solvability
 - The control problem becomes **convex optimization** with ICGNN surrogate model
 - Implicit Differentiation: Compute the gradient of implicit function, which is a form of $F(x, y) = 0$
 - Reduce the optimization search space, able to use gradient-based optimization
 - Convexity of lower-level problem** guarantees the implicit gradient is accurate

Data-Driven Bilevel Design Optimization with Input Convex Graph Neural Networks (On Going)

Target Application

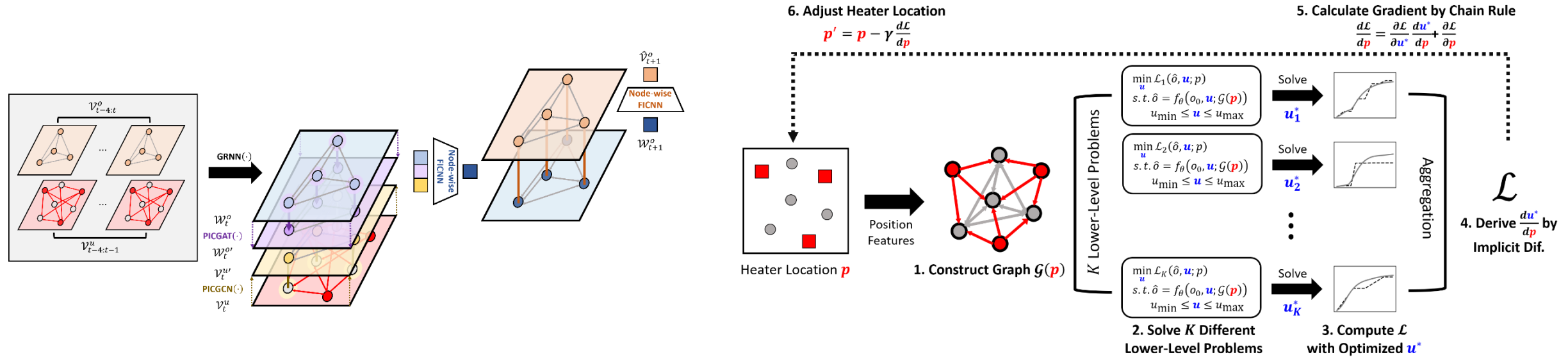


$$\begin{aligned} \min_p \quad & \frac{1}{K} \sum_{k=1}^K \sum_{t=1}^T \mathcal{L}(o_t^k, (u_{t-1}^k)^*, r_t^k) \\ \text{s.t.} \quad & (u_{0:T-1}^k)^* = \arg \min_{u_{0:T-1}^k} \sum_{t=1}^T \mathcal{L}(o_t^k, u_{t-1}^k; r_t^k), \quad \forall k \\ & \text{s.t. } o_{t+1}^k = f(o_t^k, u_t^k; p), \quad \forall t \\ & u_{\min} \leq u_t^k \leq u_{\max}, \quad \forall t \\ & p_{\min} \leq p \leq p_{\max} \end{aligned}$$

- Target System: Heat diffusion system
 - Sensors & heaters on the 2D, the dynamic of heat is governed by the diffusion equation
 - Partially observable system
- Target Problem: Heater location optimization
 - Design variable: the heater location (decision variable of the upper-level problem)
 - Operation variable: the amount of heat from heaters (decision variable of the lower-level problem)

Data-Driven Bilevel Design Optimization with Input Convex Graph Neural Networks (On Going)

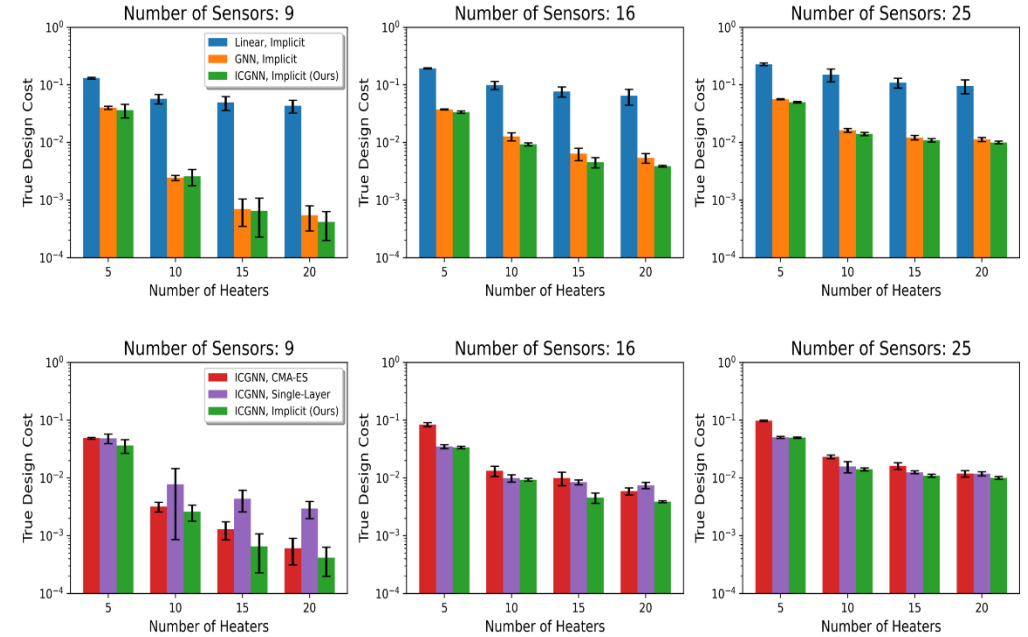
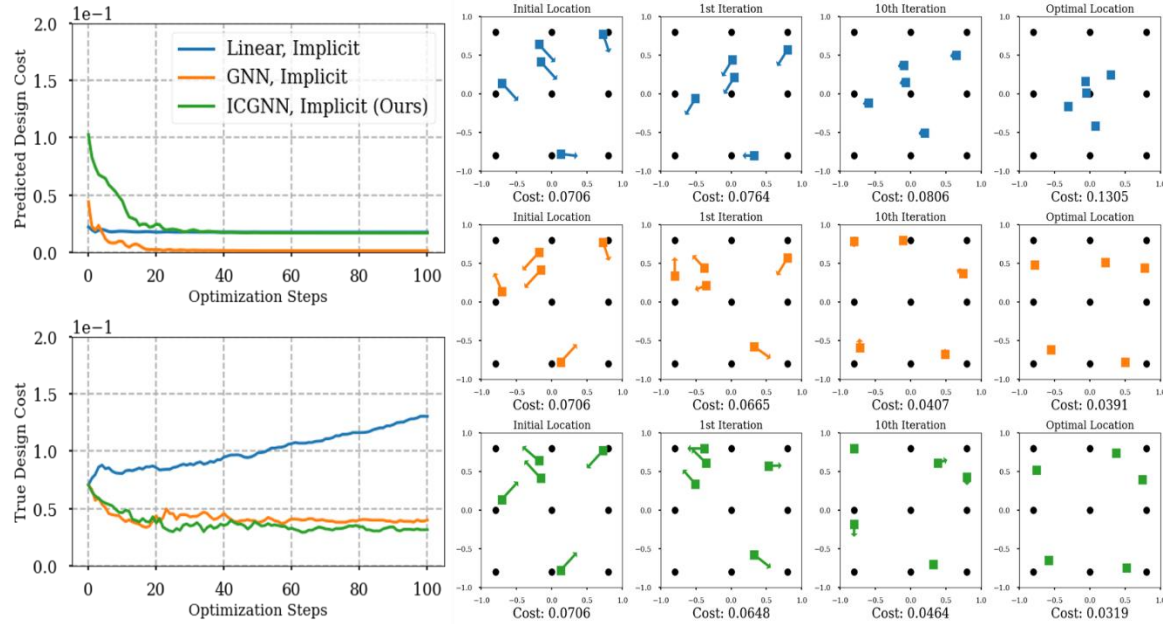
Overall procedure



- We use the partially ICGRNN surrogate model to model the true dynamic of the heat diffusion system
 - Convex path: the amount of heat from heaters, the observed heat from sensors
 - Non-convex path: the location of heaters and sensors
- Then the lower-level problem becomes the convex optimization
 - Solve the heater location optimization via the Gradient descent method

Data-Driven Bilevel Design Optimization with Input Convex Graph Neural Networks (On Going)

Experiment Results



- In the experiment, we compare the performance of our method by
 - 1) Same optimization method, but different surrogate models (Linear, GRNN)
 - 2) Same surrogate model, but different optimization method (Evolutionary-based, single-layer)
- Our method outperforms the baseline methods on various cases

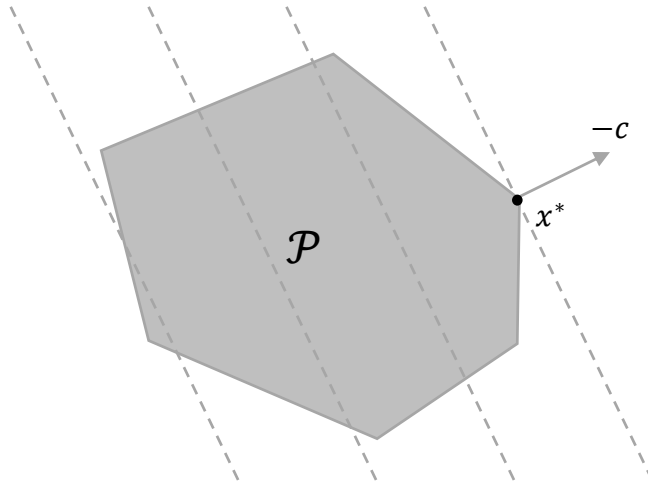
Linear Optimization

Introduction

- **Linear program (LP)** in standard form

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^T x + d \\ \text{s. t.} \quad & g_i^T x \leq h_i, \quad \forall i \in \{1, \dots, m\} \\ & a_j^T x = b_j, \quad \forall j \in \{1, \dots, p\} \end{aligned}$$

- Convex problem with **affine objective and constraint functions**
- Feasible set is a polyhedron



Introduction

Example 1.2 (diet problem)

Choose quantities $x_1, x_2, \dots, x_n$ of n foods to find cheapest healthy diet.

- One unit of food j costs c_j contains amount a_{ij} of nutrient i .
- Healthy diet requires nutrient i in quantity at least b_i .

Model the diet problem as a linear program (LP).

Introduction

Example 1.2 (diet problem)

Choose quantities $x_1, x_2, \dots, x_n$ of n foods to find cheapest healthy diet.

- One unit of food j costs c_j contains amount a_{ij} of nutrient i .
- Healthy diet requires nutrient i in quantity at least b_i .

Model the diet problem as a linear program (LP).

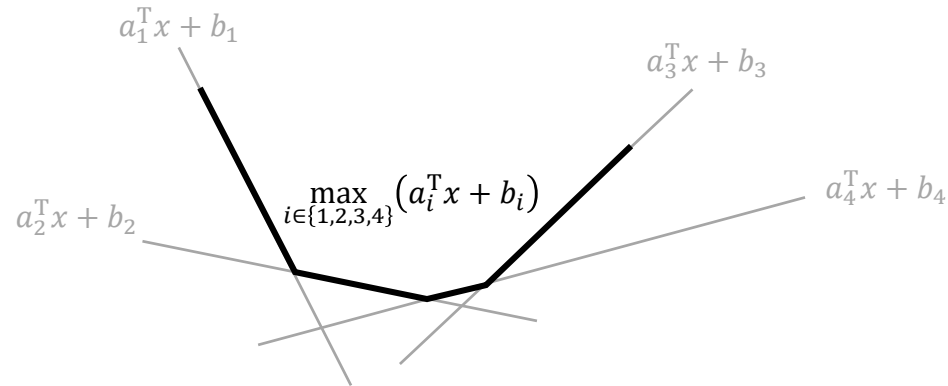
We model the diet problem as a linear program:

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}_+^{n \times 1}} \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. t.} \quad & \sum_{j=1}^N a_{ij} x_j \geq b_i, \quad \forall i \end{aligned}$$

where $\mathbf{x} = (x_j)_{j \in \{1, \dots, n\}} \in \mathbb{R}_+^{n \times 1}$, and $\mathbf{c} = (c_j)_{j \in \{1, \dots, n\}} \in \mathbb{R}_+^{n \times 1}$.

Introduction

Example 1.3 (piecewise-linear minimization)



The piecewise-linear minimization

$$\min_x \max_{i \in \{1, \dots, m\}} (a_i^T x + b_i)$$

is equivalent to an LP

$$\begin{aligned} \min_x \quad & t \\ \text{s. t.} \quad & a_i^T x + b_i \leq t, \quad \forall i \in \{1, \dots, m\} \end{aligned}$$

Introduction

Example 1.4 (Chebyshev center of a polyhedron)

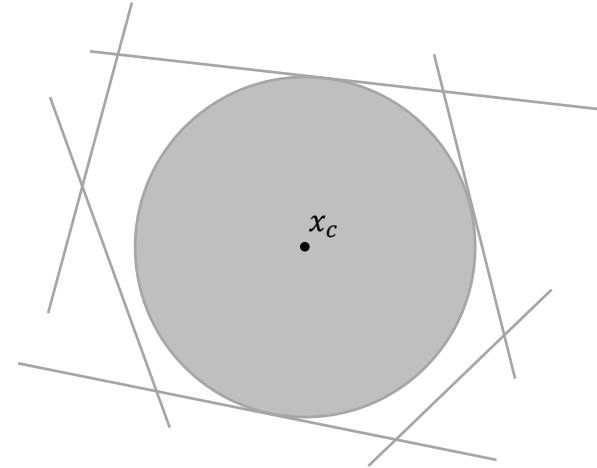
Chebyshev center of

$$\mathcal{P} = \{x \mid a_i^T x \leq b_i, \ i = 1, \dots, m\}$$

is center of largest inscribed ball

$$\mathcal{B} = \{x_c + u \mid \|u\|_2 \leq r\}$$

Model this problem as a linear program (LP).



Introduction

Example 1.4 (Chebyshev center of a polyhedron)

Chebyshev center of

$$\mathcal{P} = \{x \mid a_i^T x \leq b_i, \ i = 1, \dots, m\}$$

is center of largest inscribed ball

$$\mathcal{B} = \{x_c + u \mid \|u\|_2 \leq r\}$$

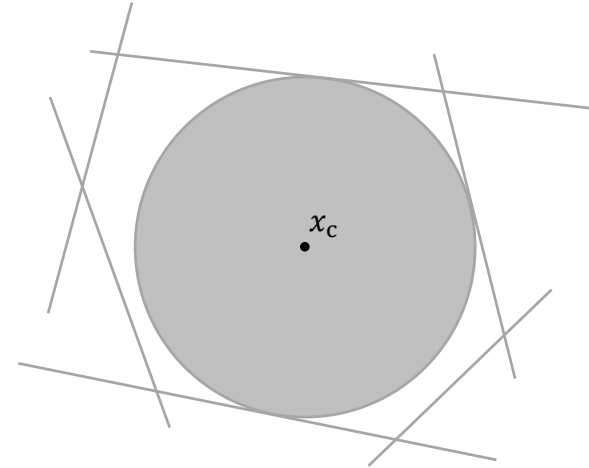
Model this problem as a linear program (LP).

The ball $\mathcal{B} \subset \mathcal{P}$ if and only if

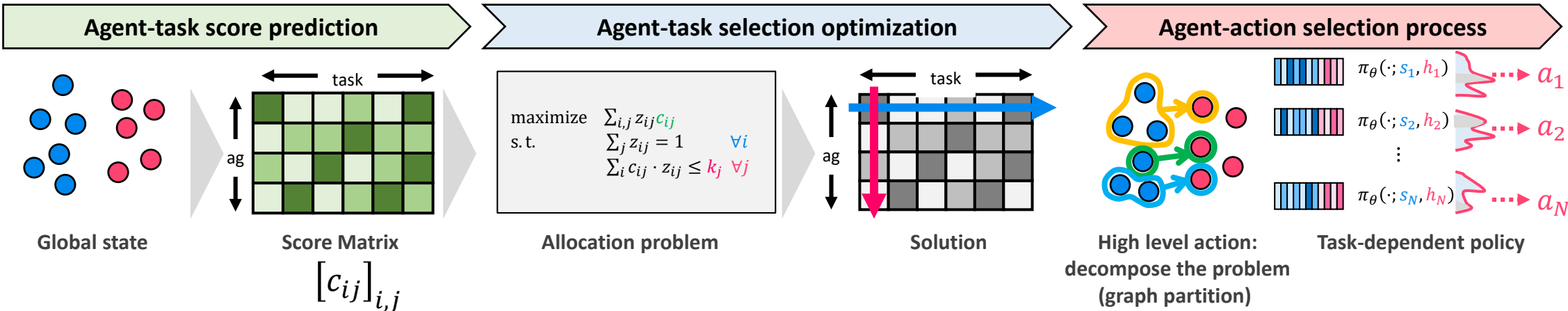
$$\sup_{\|u\|_2 \leq r} a_i^T (x_c + u) = a_i^T x_c + r \|a_i\|_2 \leq b_i, \quad \forall i \in \{1, \dots, m\}$$

Hence, x_c , and r can be determined by solving the LP:

$$\begin{aligned} & \max_{(x_c, r) \in \mathcal{P} \times \mathbb{R}_+} r \\ & \text{s. t. } a_i^T x_c + r \|a_i\|_2 \leq b_i, \quad \forall i \in \{1, \dots, m\} \end{aligned}$$



LPMARL: Hierarchical MARL with Differentiable Matching in a Layer

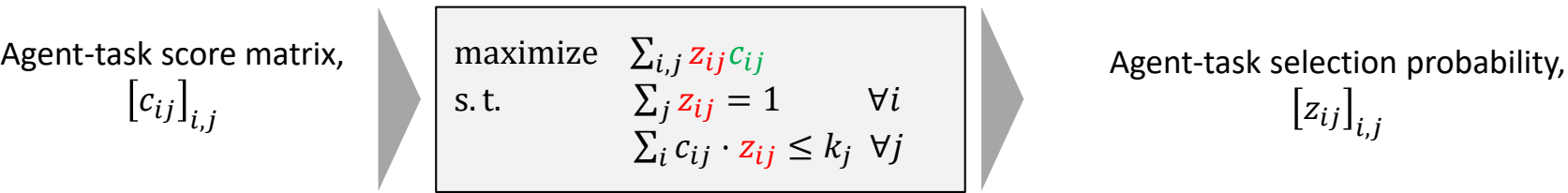


Agent-Task selection (high level policy)

- Agent-task score computation**

$c_{ij} := f_{\theta}(h_i, h_j)$, where h_i, h_j are hidden embeddings of agent i and task j given global state

- Solve optimization problem to find optimal allocation**

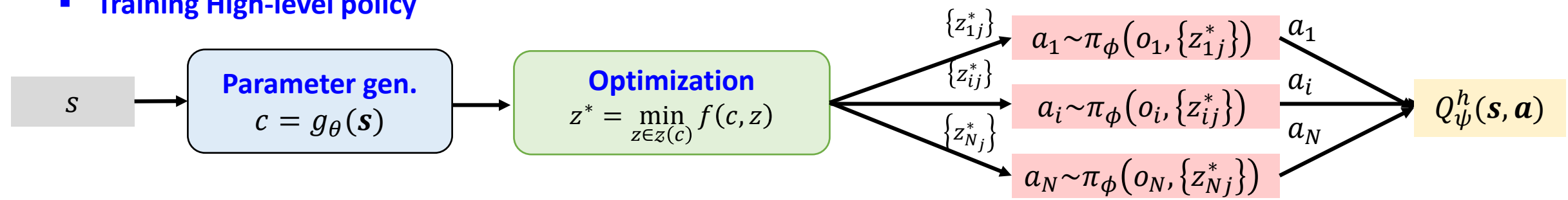


- Agent-action selection (low level policy)**

- ✓ Task-dependent policy, $\pi_{\theta}(a_i^l | h_i, h_{a_i})$
- ✓ Agents with the same task are cooperatively accomplish the task

LPMARL: Hierarchical MARL with Differentiable Matching in a Layer

Training High-level policy



Critic Update:

$$\mathcal{L}(\theta, \phi, \psi) = \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(\sum_i Q_{\psi}(s, a_i) - y \right)^2 \right] \text{ where } y = R + \gamma \sum_i Q_{\psi}(s', a'_i) \Big|_{a'_i \sim \pi_{\phi}(o_i, \{z_{ij}^*\})}$$

$$\psi \leftarrow \alpha_{\psi} \nabla_{\phi} \mathcal{L}(\theta, \phi, \psi)$$

Actor Update:

$$\nabla_{\phi} J(\phi, \theta) = \mathbb{E}_{(s,a) \sim D} \left[\nabla_{a_i} Q_{\psi}(s, a_i) \nabla_{\phi} \pi_{\phi}(o_i, \{z_{ij}^*\}) \Big|_{a_i \sim \pi_{\phi}(o_i, \{z_{ij}^*\})} \right]$$

$$\phi \leftarrow \alpha_{\phi} \nabla_{\phi} J(\phi, \theta)$$

Parameter generator update:

$$\nabla_{\theta} J(\phi, \theta) = \mathbb{E}_{(s,a) \sim D} \left[\nabla_{a_i} Q_{\psi}(s, a_i) \frac{\partial a}{\partial z^*} \frac{\partial z^*}{\partial c} \frac{\partial c}{\partial \theta} \Big|_{a_i \sim \pi_{\phi}(o_i, \{z_{ij}^*\})} \right]$$

$$\theta \leftarrow \alpha_{\theta} \nabla_{\theta} J(\phi, \theta)$$

$z^* = z^*(c) = z^*(g_{\theta}(s)) : \theta \text{ implicitly induces } z^*$

$\frac{\partial z^*}{\partial c}$ can be found by computing Jacobian of KKT equalities

Quadratic Optimization

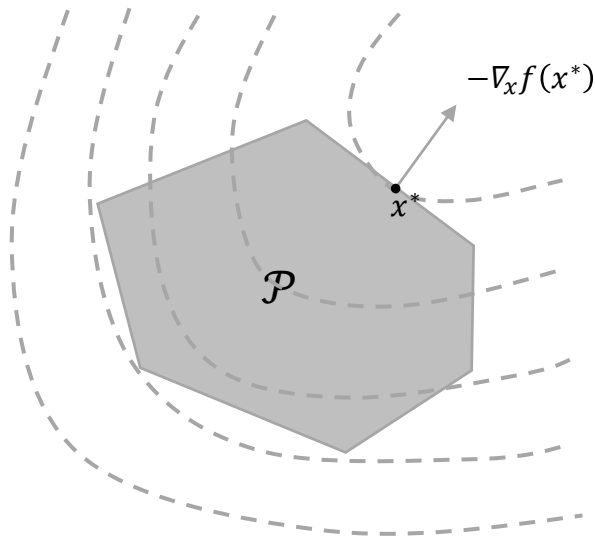
Introduction

- **Quadratic program (QP)** in standard form

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & \frac{1}{2} x^T \mathbf{P} x + q^T x + r \\ \text{s. t.} \quad & g_i^T x \leq h_i, \quad \forall i \in \{1, \dots, m\} \\ & a_j^T x = b_j, \quad \forall j \in \{1, \dots, p\} \end{aligned}$$

where $\mathbf{P}$ is positive semidefinite. So, objective is convex quadratic.

- Minimize a **convex quadratic function over a polyhedron**.



Introduction

- Least-squares with linear constraints

$$\begin{aligned} \min_{x \in \mathbb{R}^n} & \|Ax - b\|_2^2 \\ \text{s. t. } & l \leq x \leq u \end{aligned}$$

- Linear program with random cost

$$\begin{aligned} \min_{x \in \mathbb{R}^n} & \bar{c}^T x + \gamma x^T \Sigma x \\ \text{s. t. } & g_i^T x \leq h_i, \quad \forall i \in \{1, \dots, m\} \\ & a_j^T x = b_j, \quad \forall j \in \{1, \dots, p\} \end{aligned}$$

- c is random vector with mean $\bar{c}$ and covariance Σ .
- $c^T x$ is random variable with mean $\bar{c}^T x$ and variance $x^T \Sigma x$.
- $\gamma > 0$ is risk aversion parameter; controls the trade-off between expected cost and variance (risk)

Quadratically constrained quadratic program (QCQP)

- Quadratically constrained quadratic program (QCQP) in standard form

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & \frac{1}{2} x^T \mathbf{P}_0 x + q_0^T x + r_0 \\ \text{s. t.} \quad & \frac{1}{2} x^T \mathbf{P}_i x + q_i^T x + r_i \leq 0, \quad \forall i \in \{1, \dots, m\} \\ & a_j^T x = b_j, \quad \forall j \in \{1, \dots, p\} \end{aligned}$$

where $\mathbf{P}_i$ is positive semidefinite. That is, objective and constraints are convex quadratic.

- If P_i is positive definite for all $i \in \{1, \dots, m\}$, then feasible region is intersection of m ellipsoids and an affine set.

Robust linear programming

- The parameters in optimization problems are often uncertain, that is, in an LP:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^T x + d \\ \text{s. t.} \quad & g_i^T x \leq h_i, \quad \forall i \in \{1, \dots, m\} \end{aligned}$$

- There can be uncertainty in c , g_i , and h_i .
- Two common approaches to handling uncertainty in g_i :

- ✓ **Deterministic model**: constraints must hold for all $g_i \in \mathcal{E}_i$

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^T x + d \\ \text{s. t.} \quad & g_i^T x \leq h_i, \quad \forall g_i \in \mathcal{E}_i, \quad \forall i \in \{1, \dots, m\} \end{aligned}$$

- ✓ **Stochastic model**: g_i is random variable, constraints must hold with probability η

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^T x + d \\ \text{s. t.} \quad & \mathbf{P}(g_i^T x \leq h_i) \geq \eta, \quad \forall i \in \{1, \dots, m\} \end{aligned}$$

Case Study: Wind Farm Layout Optimization

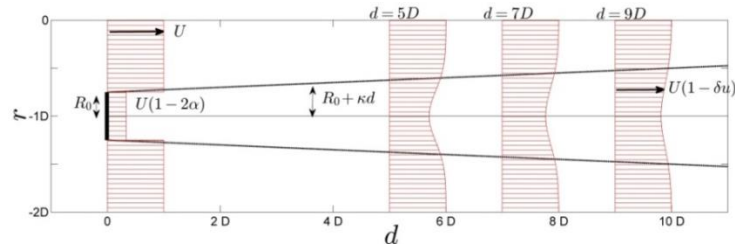


- **Wind farm power efficiency degrades as its size increases**
 - ✓ The largest wind farm is the London array in the UK, which produces 630 MW of power with 175 wind turbines
 - ✓ due to wake interference, the power efficiency of a wind farm decreases as the number of wind turbines at a given site increases.

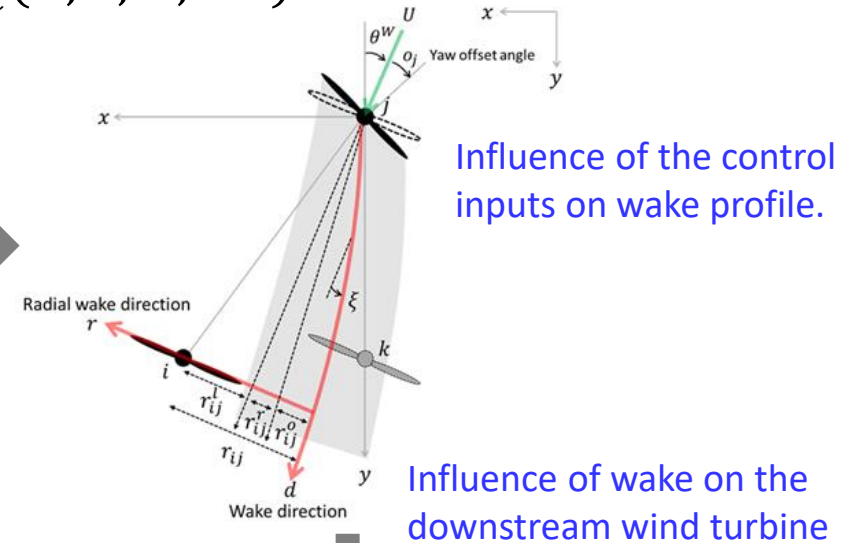
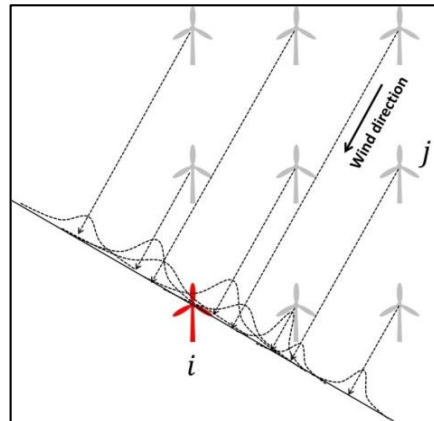
Case Study: Wind Farm Layout Optimization

Construction of wind turbine power function $P_i(\alpha, \mathbf{o}; U, \theta^W)$

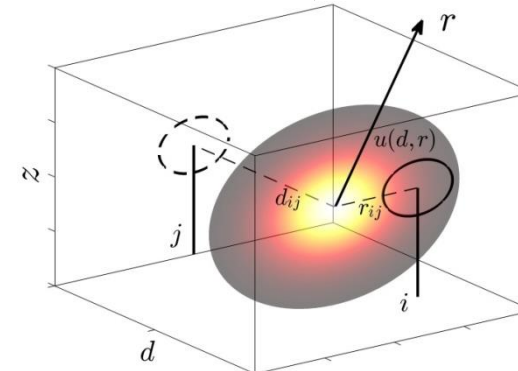
Influence of wind turbine on the wind speed profile



Influence of the multiple upstream turbines on power at i



Influence of wake on the downstream wind turbine



- The power of wind turbine i due to all the wind turbines:

$$P_i(\alpha, \mathbf{o}; U, \theta_W) = \frac{1}{2} \rho A C_P(\alpha_i, o_i) \bar{u}_i^3(\alpha_{-i}, \mathbf{o}_{-i}; U, \theta_W)$$

- $C_P(\alpha_i, o_i) = 4\alpha_i(\cos(\beta o_i) - \alpha_i)^2$

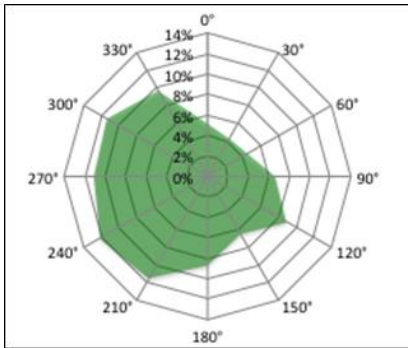
Wind speed due to other control actions

Case Study: Wind Farm Layout Optimization

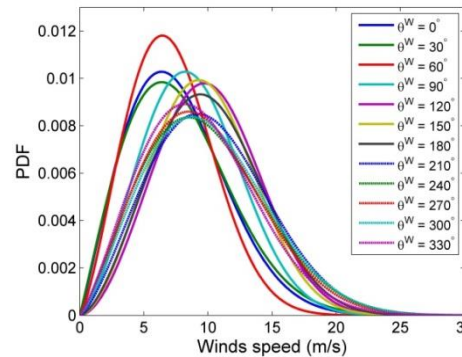
Maximizing the expected wind farm power

- Wind direction distribution is obtained from the target wind farm site.
- Wind speed distribution in each wind direction bin is modeled as Weibul distribution.
- A joint probability mass function for the wind direction and the speed is used to construct the expected wind farm power function, which is maximized by SCP.

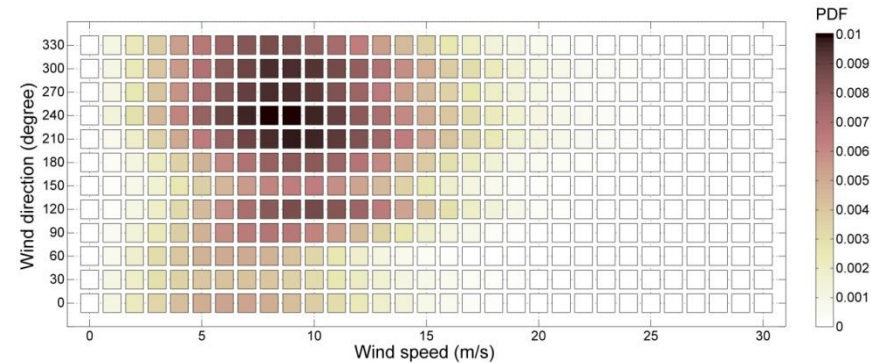
$$f(\mathbf{l}) = E \left[\sum_{i=1}^N P_i(\mathbf{l}; U, \theta^W) \right] = \sum_{k=1}^{N_\theta} \sum_{j=1}^{N_U} \sum_{i=1}^{N_T} P_i(\mathbf{l}; U_j, \theta_k^W) \Pr(U_j | \theta_k^W) \Pr(\theta_k^W)$$



$\Pr(\theta_k^W)$



$\text{PDF}(U_j | \theta_k^W) \rightarrow \Pr(U_j | \theta_k^W)$



A joint probability mass function
 $\Pr(U_j, \theta_k^W) = \Pr(U_j | \theta_k^W) \Pr(\theta_k^W)$

Case Study: Wind Farm Layout Optimization

Maximizing the expected wind farm power

$$\begin{aligned}\text{maximize}_{\mathbf{l}} \quad f(\mathbf{l}) &= \mathbb{E} \left[\sum_{i=1}^N P_i(\mathbf{l}; U, \theta^W) \right] \\ &= \int_{\theta^W} \int_U \sum_{i=1}^N P_i(\mathbf{l}; U, \theta^W) p(U, \theta^W) dU d\theta^W \\ &\approx \sum_{k=1}^{N_\theta} \sum_{j=1}^{N_U} \sum_{i=1}^N P_i(\mathbf{l}; U_j, \theta_k^W) \Pr(U_j, \theta_k^W)\end{aligned}$$

$$\begin{aligned}\text{subject to} \quad & \|l_i - l_j\|_2 \geq 5D \quad \text{for } i, j = 1, \dots, N_T \quad (i \neq j) \\ & \underline{\mathbf{c}} \leq \mathbf{Cl} \leq \bar{\mathbf{c}}\end{aligned}$$

- $\mathbf{l}$: locations of N wind turbines
- $P_i(\mathbf{l}; U, \theta^W)$: Power of wind turbine i given wind speed U and wind direction θ^W
- $\Pr(U_j, \theta_k^W)$: Probability mass function

Case Study: Wind Farm Layout Optimization

Algorithm

Solving cooperative control problem using sequential quadratic programming

$$\underset{\mathbf{l}}{\text{maximize}} \quad f(\mathbf{l}) = \mathbb{E} \left[\sum_{i=1}^N P_i(\mathbf{l}; U, \theta^W) \right] = \sum_{k=1}^{N_\theta} \sum_{j=1}^{N_U} \sum_{i=1}^{N_T} P_i(\mathbf{l}; U_j, \theta_k^W) \Pr(U_j | \theta_k^W) \Pr(\theta_k^W)$$

$$\text{subject to} \quad \|\mathbf{l}_i - \mathbf{l}_j\|_2 > 5D \quad \text{for } i, j = 1, \dots, N_T \quad (i \neq j)$$

$$\underline{\mathbf{c}} \leq \mathbf{C}\mathbf{l} \leq \bar{\mathbf{c}}$$

- $\mathbf{l}^k$: the estimated solution at the k th iteration
- Function value $f^k = f(\mathbf{l}^k)$,
- Analytically evaluated gradient $\mathbf{g}^k = \nabla f(\mathbf{l}^k)$
- Approximated Hessian $\mathbf{B}^k$ at the solution $\mathbf{l}^k$ of the k th iteration,

$$\mathbf{l}^{k+1} = \mathbf{l}^k + a(\hat{\mathbf{l}} - \mathbf{l}^k)$$

a is the step size ensuring sufficient increase in $f(\mathbf{l})$

Solution $\hat{\mathbf{l}}$ of QP

Approximated
Quadratic program (QP)
at the current solution $\mathbf{l}^k$

$$\begin{aligned} \underset{\mathbf{l}}{\text{maximize}} \quad & \tilde{f}(\mathbf{l}) := f(\mathbf{l}^k) + \nabla f(\mathbf{l}^k)^T (\mathbf{l} - \mathbf{l}^k) + 1/2 (\mathbf{l} - \mathbf{l}^k)^T \mathbf{B}^{(k)} (\mathbf{l} - \mathbf{l}^k) \\ \text{subject to} \quad & \left(\mathbf{l}_i^{(k)} - \mathbf{l}_j^{(k)} \right)^T (\mathbf{l}_i - \mathbf{l}_j) > 4D \left\| \mathbf{l}_i^{(k)} - \mathbf{l}_j^{(k)} \right\|_2 \quad \text{for } i, j = 1, \dots, j \quad (i \neq j) \\ & \underline{\mathbf{c}} \leq \mathbf{C}\mathbf{l} \leq \bar{\mathbf{c}} \\ & \mathbf{l} \in T^{(k)} = \{\mathbf{l} \mid \|\mathbf{l} - \mathbf{l}^k\| < \rho^{(k)}\} \end{aligned}$$

Case Study: Wind Farm Layout Optimization

Algorithm

- Using the analytically evaluated gradient $\nabla f(\mathbf{l}^{(k)})$ and the approximated hessian $B^{(k)}$ at the current estimate $\mathbf{l}^{(k)}$, the SCP approximate the original problem into:

$$\begin{aligned} & \underset{\mathbf{l}}{\text{maximize}} && \tilde{f}(\mathbf{l}) := f(\mathbf{l}^k) + \nabla f(\mathbf{l}^k)^T (\mathbf{l} - \mathbf{l}^k) + 1/2(\mathbf{l} - \mathbf{l}^k)^T B^{(k)} (\mathbf{l} - \mathbf{l}^k) \\ & \text{subject to} && \left(\mathbf{l}_i^{(k)} - \mathbf{l}_j^{(k)} \right)^T (\mathbf{l}_i - \mathbf{l}_j) > 4D \left\| \mathbf{l}_i^{(k)} - \mathbf{l}_j^{(k)} \right\|_2 \text{ for } i, j = 1, \dots, J \ (i \neq j) \\ & && \underline{\mathbf{c}} \leq \mathbf{C}\mathbf{l} \leq \overline{\mathbf{c}} \\ & && \mathbf{l} \in T^{(k)} = \{ \mathbf{l} \mid \|\mathbf{l} - \mathbf{l}^k\| < \rho^{(k)} \} \end{aligned}$$

- Objective function is approximated as a **quadratic function**.
- Inter-minimum distance constrain is **linearized (convexified)**.
- The constraint $\mathbf{l} \in T^{(k)} = \{ \mathbf{l} \mid \|\mathbf{l} - \mathbf{l}^k\| < \rho^{(k)} \}$, referred to as a trust region, is additionally imposed to guarantee that the approximated function is close to the true function



The approximated Convex problem (Quadratic programming) can be efficiently solved by using an efficient Semi Definite Programming (SDP) solver.

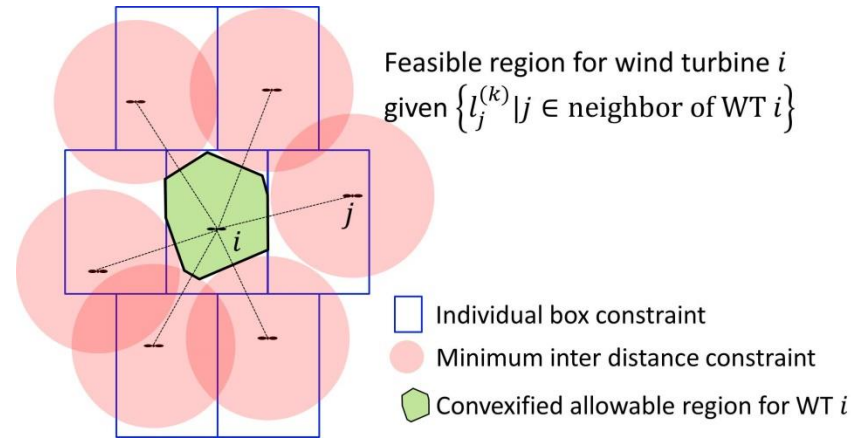


The solution $\tilde{\mathbf{l}}$ is used as the next iteration point : $\mathbf{l}^{k+1} = \tilde{\mathbf{l}}$ given that $\tilde{\mathbf{l}}$ improves the objective function

Case Study: Wind Farm Layout Optimization

Algorithm

Linearization of the inter-minimum distance constraints:



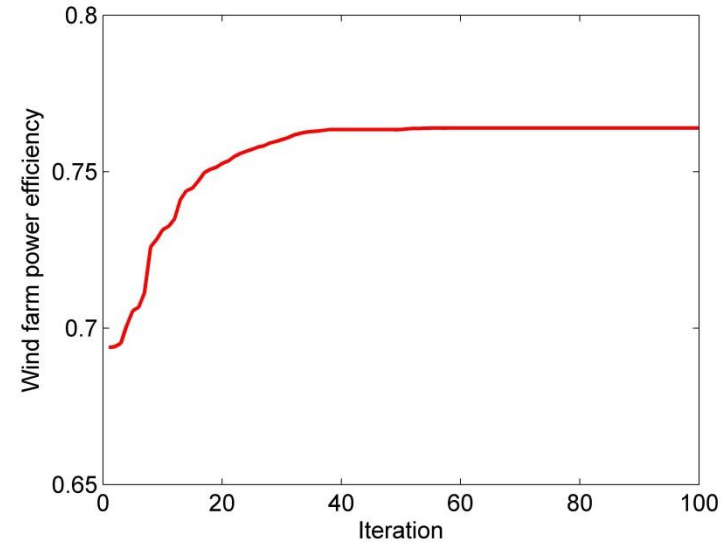
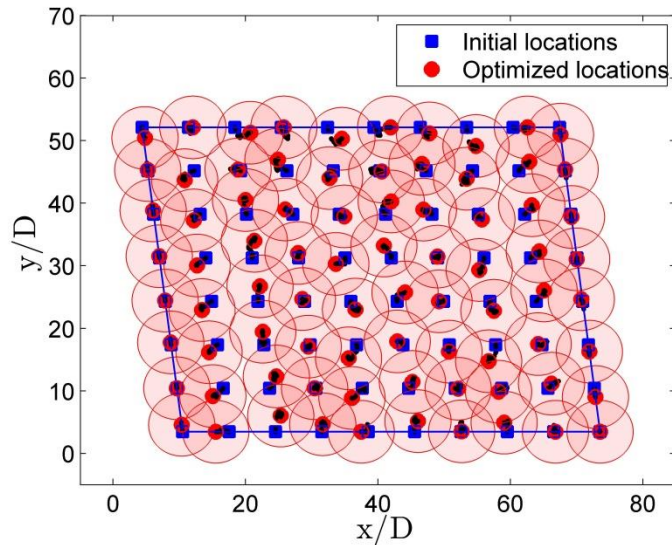
Overall algorithm :

Algorithm 1 Wind farm layout optimization using SCP

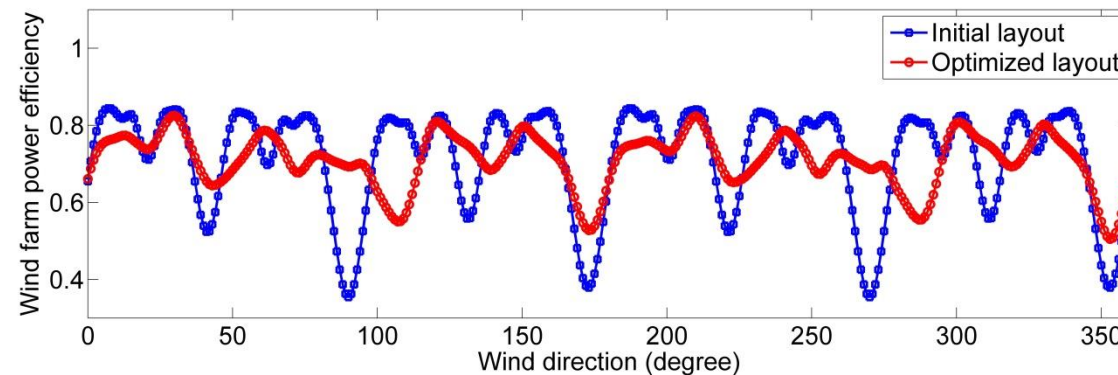
- 1: initialize (choose) $l^{(0)}$
 - 2: initialize the trust region $T^{(0)} = \{l \mid \|l - l^{(0)}\| < \rho^{(0)}\}$
 - 3: **while** $\|l^{(k)} - l^{(k-1)}\|_2 < \epsilon$ **do**
 - 4: find $\tilde{l}$ maximizing $\tilde{f}(l)$ and satisfying the constraints
 - 5: **if** $\frac{f(l^{(k)}) - f(\tilde{l})}{f(l^{(k)}) - \hat{f}(\tilde{l})} \geq \alpha$ **then**
 - 6: update the solution $l^{(k+1)} = \tilde{l}$
 - 7: increases the size of trust region $\rho^{(k+1)} = \beta^{\text{success}} \rho^{(k)}$
 - 8: **else**
 - 9: reject the current solution $l^{(k+1)} = l^{(k)}$
 - 10: decrease the size of trust region $\rho^{(k+1)} = \beta^{\text{fail}} \rho^{(k)}$
 - 11: **end if**
 - 12: increment $k \leftarrow k + 1$
 - 13: **end while**
-

Case Study: Wind Farm Layout Optimization

Results



- Wind turbine locations are relocated so as to reduce the successive wake interference, resulting in a significant increase in a wind farm power efficiency.



- In optimized layout, the level of fluctuation in the wind farm power is reduced compared to initial layout.